

WeBe Band: On-Device Low-Latency Motion Recognition

Edwin Kay¹, Wei-Hsiang Chu², Junwei Lu³, Jady N. Miyahira³

¹Healthtile Inc., Cupertino, CA, U.S.A.

²Department of Electrical and Computer Engineering, University of California, Davis, Davis, CA, U.S.A.

³Department of Computing and Data Science, University of California, Berkeley, Berkeley, CA, U.S.A.

Email: ekay@healthtile.io, ruchu@ucdavis.edu, {jw.l, jady.n.miyahira}@berkeley.edu

Abstract—The use of tiny devices capable of low-latency gesture recognition is gaining momentum in everyday human-computer interaction and especially in medical monitoring fields. Embedded solutions such as fall detection, rehabilitation tracking, and patient supervision require fast and efficient tracking of movements while avoiding unwanted false alarms. This study presents an efficient solution on how to build very efficient motion-based models only using triaxial accelerometer sensors. We explore the capability of the AutoML pipelines to extract the most important features from the data segments. This approach also involves training multiple lightweight machine learning algorithms using the extracted features. We use *WeBe Band*, a multi-sensor wearable device that is equipped with a powerful enough MCU to effectively perform gesture recognition entirely on the device. Of the models explored, we found that the neural network provided the best balance between accuracy, latency, and memory use. Our results also demonstrate that reliable real-time gesture recognition can be achieved in *WeBe Band*, with great potential for real-time medical monitoring solutions that require a secure and fast response time.

Index Terms—edge computing, machine learning, AutoML, medical wearables, gesture recognition

I. INTRODUCTION

Motion recognition capability is one of the prominent applications that many wearable devices need to have for activity tracking. In medical settings, such as hospitals and clinics, accurate, continuous, and reliable monitoring is essential for applications ranging from patient rehabilitation to elderly fall detection [1], and device adherence. All such applications pose real challenges when deployed in sensitive environments [2]. Medical environments rapidly evolve, and conventional camera-based methods [3] are prone to mistakes in recognizing complicated scenarios and unpredictable environments. Most embedded motion detection algorithms rely on 6-channel gyroscope and accelerometer data (IMU). However, here we focus on accelerometer-based sensing to provide a more compact, non-invasive alternative that captures subtle motion dynamics at low cost.

Developing any embedded motion detection solution requires extra attention. Many approaches rely on handcrafted features extracted from 6-axis IMU signals and neglect the richer information that might be available in multi-axis accelerometer signals [4]. Limiting the signals to accelerometers does not necessarily indicate that there is not enough inherent

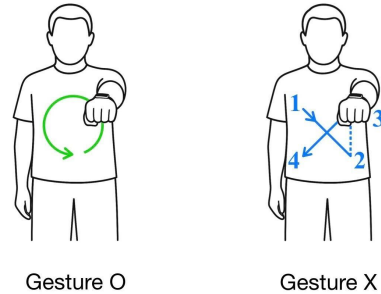


Fig. 1. Illustration of the “X” and “O” gestures from the perspective of an external observer. The numbers indicate the sequential movement steps performed by the participant.

information to operate a reliable motion recognition system. Improper data splitting or validation strategies can cause leakage, leading to overly optimistic performance estimates. Any embedded model needs to be tested using a simulator or on the device to validate its performance in the real setup.

We propose a generalizable gesture recognition framework using 3-axis accelerometer signals on *WeBe Band*, a recently introduced watch-like wearable platform for continuous motion recognition [5]. At its core, *WeBe Band* is equipped with an ARM Cortex-M4F microcontroller (*nRF52840*, 64 MHz), with Flash and RAM sizes of 1 MB and 256 KB, respectively [5].

In this study, we leverage the triaxial accelerometer signals of *WeBe Band* to build a gesture detection algorithm, and use AutoML to extract the most informative features and augment the data to increase model robustness against unforeseen factors [6]. We train and evaluate lightweight models, i.e. Neural Networks (NN), Random Forests (RF), Bonsai, and Pattern Matching Engine (PME), through device-level profiling. All developed gesture recognition models are deployed and executed directly on the device, with low-latency responses that are critical for applications such as fall detection and urgent medical alerts. On the other hand, raw sensor data never leaves the device; hence, enhancing privacy and security in super-sensitive medical environments.

II. METHODOLOGY

A. Data

For this study, we exclusively use the integrated triaxial accelerometer sensor in *WeBe Band*. Gesture data were collected at a sampling rate of 25 Hz.

B. Data Collection

Timeseries data were collected using the *WeBe Band* worn on the left wrist. Five participants were recruited for the study. Each participant completed five recording sessions for each gesture. In each session, they performed approximately ten gestures separated by rest intervals of 3-5 seconds.

We considered two significantly distinct gestures (illustrated in Fig. 1):

- **“X” Gesture:** The participant moves the hand in a large “X” shape in the air, following the following sequence: starting from the center, (1) to the top right, (2) to the bottom left, (3) to the top left, (4) to the bottom right, and finally returning to the center.
- **“O” Gesture:** The participant makes a clockwise circular motion forming the letter “O”.
- **Random Gestures:** The participants also performed unconfined hand movements, excluding deliberate shapes “X” and “O”, to improve model robustness against non-target motions and to lower misclassification rates.

Representative accelerometer signals for “X,” “O,” and random gestures are shown in Fig. 2.

C. Data Annotation

We segmented all gestures, each containing $\gtrsim 100$ samples (~ 4 seconds), and labeled them manually and semi-automatically. In some cases, we used the *WeBe Band* event button to mark the exact moments of gestures, and used a custom script to extract and label segments. We manually verified the quality of all labels.

D. Data Augmentation

To ensure balanced class representation between “X” and “O” gestures, we applied data augmentation crafted for time-series data:

- **Temporal Shifting:** In real-time deployment, data segments may be shifted depending on the state of the signal segmenter. Consequently, each label was randomly shifted by 7–15 samples to generate additional instances.
- **Amplitude Scaling:** Signals were scaled by $\pm 10\%$ to simulate slight variations in gesture intensity and incorporate user-to-user variance in motion amplitude.
- **Time Stretch:** To address the variance of motion speed, signals were stretched by $\pm 5\%$.

III. ML MODEL EXPLORATION

A. AutoML

To identify promising features extracted from each segment, we first applied automated pipeline search using the open-source *Piccolo AI* toolkit [7]. It is designed to build and deploy

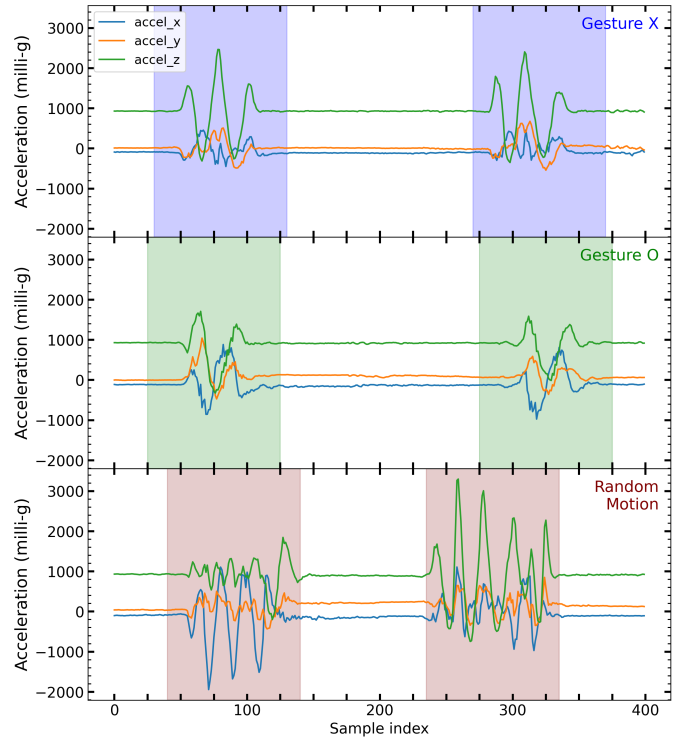


Fig. 2. Representative tri-axial accelerometer signals for the three gesture classes, highlighted in color. From top to bottom (a) “X,” (b) “O,” and (c) Random Gestures. Each panel shows two instances of 4-sec gestures (highlighted in color), exhibiting the characteristic motion patterns that distinguish the classes.

ML models on embedded and IoT devices such as *WeBe Band*. A key feature of this framework is its Automated Machine Learning (AutoML), that significantly helps researchers to systematically explore a large space of features, models, and hyperparameters. In a single run, the system evaluates a large number of candidates and produces a small set of models that perform well, along with the features that contributed to their success. *Piccolo AI* employs a genetic algorithm that evaluates an evolving set of extractors and classifiers based on accuracy, latency, and memory footprint. We mainly rely on this approach to find effective features, while it also offers insights good model candidates for embedded deployment.

B. Feature Selection

Due to the random nature of AutoML, each run can return a different set of features/models. Hence, we repeated the process several times. For each iteration, we documented the feature sets used by best performing models, and documented the features used by these models. The feature pool we explored for the AutoML process included statistical descriptors (mean, variance, kurtosis), signal amplitude measures (peak-to-peak and min-max distance indicators, zero-crossing rates), and cross-axes feature fusion. The most common identified subset is summarized in Table I, drawn from the feature sets of the top 5 well-performing models in all 4 iterations. To investigate the discriminative potential of these features,



Fig. 3. t-SNE visualization of the selected feature clusters for different gesture classes on the *WeBe Band* dataset.

TABLE I
TOP FEATURE CANDIDATES FOUND BY AUTOML.

Feature (with description)	$accel - X$	$accel - Y$	$accel - Z$
Mean	✓	✓	-
Variance	-	✓	✓
Kurtosis	-	✓	-
25th Percentile	-	✓	-
75th-25th Percentile Range	✓	✓	-
Negative Zero Crossings	✓	✓	✓
Global Min and Max Sum	✓	-	-
Median Difference of Cross Axes	✓	✓	✓
Min-Max Distances	✓	✓	✓
Global Peak-to-peak Distance	-	✓	✓

we used the t-SNE dimensionality reduction algorithm [8]. Evidently, the selected features are capable of supporting reasonable classification (see Fig. 3).

C. Model Generation

We re-trained each individual algorithm on the best identified features. To maximize the models performances, we fine-tuned their hyperparameters.

We trained four ML algorithms as described below.

- **Pattern Matching Engine (PME):** PME is a distance-based classifier that compares new inputs with the stored prototypes using a distance metric, e.g., L_1 or L -sub metrics [9]. Each prototype is associated with an *Area of Influence (AIF)*, defining the maximum distance range within which a sample can be considered similar to that prototype. In our experiments, we set the maximum (minimum) allowed AIF to 400 (25), to balance specificity and robustness while avoiding overfitting. The maximum number of dynamically added neurons to span the entire feature space is set to 128.

TABLE II
CLASS-SPECIFIC PERFORMANCE METRICS ON TEST SET (%)

Gesture Class	PME*		RF*		Bonsai		NN*	
	Recall	Prec.	Recall	Prec.	Recall	Prec.	Recall	Prec.
O Gesture	99.1	98.2	95.2	95.2	95.2	97.5	99.1	96.5
X Gesture	99.5	93.3	96.8	96.8	98.9	93.1	99.5	98.6
Random Motion	96.5	99.7	98.1	98.1	96.4	100	97.1	100
Total Accuracy	97.7		97.6		97.1		98.1	
Macro F1-score	97.7		96.8		96.8		98.6	

* PME = Pattern Matching Engine, RF = Random Forest, NN = Neural Network. Prec. = Precision, Recall = Recall (Precision)

- **Random Forest (RF):** Known as one of the strongest classical algorithms for motion recognition. Using multiple decision trees during training, it outputs the class by taking the majority vote among all predictions. This approach is usually well-suited for embedded applications and inherently exhibits robustness against overfitting, while being capable of effectively handling noisy sensor data. We trained a forest of 40 decision trees, each with a maximum depth of 7. These parameters were tuned to achieve the highest classification accuracy.
- **Bonsai:** The Bonsai tree optimizer is specifically designed for efficient inference on microcontrollers [10]. It captures non-linear decision boundaries while maintaining a small footprint. Bonsai linearly maps high-dimensional feature spaces into lower dimensions in combination with a shallow decision tree. We optimized Bonsai with a set of fine-tuned hyperparameters, including a projection dimension of 13, tree depth of 4, and kernel width $\sigma = 0.3$. The model was trained for 500 epochs with a batch size of 32 and a learning rate of 0.01.
- **Neural Network (NN):** We trained several simple fully connected neural networks (NNs) on the extracted features. The network consists of four dense layers of varying sizes. After experimenting with a few architectures, we found the best-performing layer architecture of 16, 16, 8, and 4 neurons. Each NN was trained for 10 epochs with a batch size of 32 using the Adam optimizer, learning rate = 0.0015, dropout rate of 0.1 to prevent overfitting, and batch normalization. To reduce the false-positive rate, the output was post-processed with a conservative 60% confidence threshold, mapping low-confidence predictions to “Unknown”.

IV. SUMMARY OF RESULTS

We divided our data into train-validation-test sets with a fraction of 60-20-20%. During the training and fine-tuning process, we class-balanced the data according to Section II-D. To realistically assess model misclassifications, we did not balance the test set to better reflect deployment conditions. Each model was assessed based on classification accuracy, F1-score, latency, and memory usage. We evaluated each model in class-specific *Precision* and *Recall*. Precision measures the percentage of generated predictions that are correct, while Recall measures the proportion of each ground-truth gesture

TABLE III
NEURAL NETWORK CONFUSION MATRIX FOR THE TEST SET (ROWS =
GROUND TRUTH, COLUMNS = PREDICTIONS).

	O	X	Rand	UNC	Support	Recall (%)
True O	113	0	0	1	114	99.1
True X	0	223	0	1	224	99.5
True Rand	4	3	482	7	496	97.1
Predicted Precision (%)	117 96.5	226 98.6	482 100	9 —	834 —	— Acc = 98.1

* Rand = Random gestures, UNC = Uncertain

TABLE IV
PERFORMANCE COMPARISON OF MODELS AND AUTOML OUTCOMES ON
THE WEBe BAND.

ML Model	Accuracy	F1-score	Latency	SRAM+STACK	FLASH
	(%)	(%)	(ms)	(KB)	(KB)
AutoML* (NN**)	97.5	97.7	1.2	7.7	18.0
Random Forest	97.6	96.8	4.6	2.6	20.1
Bonsai	97.1	96.8	5.7	3.1	17.5
PME	97.7	97.7	4.6	2.7	9.8
Neural Network (NN)	98.1	98.6	1.2	7.7	18.0

* AutoML = best pipeline identified. ** NN = Neural Network.

that is recognized correctly. These metrics are summarized in Table II. Having a rigorous method to select the best features and hyperparameters tuning, their performance turned out to be insignificantly different, with the neural network performing the best overall. Table III shows the confusion matrix of the neural network model in the test set. The model clearly recognizes target gestures and distinguishes them from random motions. Notably, only 3% of the random ground truth segments were incorrectly mapped to other gestures. The overall performances of the investigated models are summarized in Table IV. Contrary to common assumptions, the neural network achieved both the highest accuracy and the lowest latency (1.2 ms). Although Random Forests, Bonsai, and PME were slower, they remain viable alternatives in low-memory scenarios.

A. Model Profiling on the WeBe Device

We profiled each model on the *WeBe Band* to evaluate their real-time performance. *Piccolo AI* provides an estimate of the memory usage of each model and also facilitates quantization-aware training of neural networks, as well as quantization using TensorFlow Lite for deployment on microcontrollers. We implemented a few profiling units in the *WeBe* firmware. To reduce profiling overhead, we leveraged the CoreSight debug block of the Data Watchpoint and Trace (DWT) unit. A 32-bit cycle counter was sampled immediately before and after each model call. Using the clock frequency of 64 MHz, we converted the cycle difference into microseconds to precisely measure latency. Table IV presents the results. Surprisingly, the neural network achieves the lowest latency while still delivering the best overall performance, which highlights the benefits of our feature selection procedure. The PME model also appears promising for memory constraint systems.

V. CONCLUSION AND FUTURE WORK

In this study, we demonstrated the feasibility of performing accurate low-latency gesture recognition directly on *WeBe Band* utilizing only triaxial accelerometer sensors. We built compact machine learning models with carefully optimized selected features. All of our fine-tuned models achieved robust on-device classification and performed satisfactorily when deployed on the device. We found that among the models explored, neural networks not only achieved the lowest latency (< 2 milliseconds), outperforming traditional machine learning approaches such as ensemble decision trees and distance-based classifiers. All models exhibited an overall accuracy better than 95%, with neural networks showing promise to achieve higher performance. This result highlights that with the right features and efficient hyperparameter tuning, all of the explored models can remain both compact and fast on microcontrollers.

Beyond gesture recognition, the *WeBe Band* offers a versatile platform for medical and everyday health monitoring, with potential applications such as rehabilitation tracking, fall detection, and continuous patient supervision. Future work involves integrating additional biomarkers, such as PPG, as well as developing model triggering mechanisms to enable longer battery life and always-on operation when needed.

ACKNOWLEDGMENT

We gratefully acknowledge the *WeBe Band* team for their support in deploying our solutions on the device.

REFERENCES

- [1] S. Nooruddin, M. M. Islam, F. A. Sharna, *et al.*, “Sensor-based fall detection systems: a review,” *Journal of Ambient Intelligence and Humanized Computing*, vol. 13, pp. 2735–2751, 2022, doi: 10.1007/s12652-021-03248-z.
- [2] M. Shoaib, S. Bosch, O. D. Incel, H. Scholten, and P. J. Havinga, “Complex human activity recognition using smartphone and wrist-worn motion sensors,” *Sensors*, vol. 16, no. 4, p. 426, 2016.
- [3] J. Wachs, M. Kölsch, H. Stern, and Y. Edan, “Vision-based hand-gesture applications,” *Communications of the ACM*, vol. 54, no. 2, pp. 60–71, 2011.
- [4] S. Zhao, H. Cai, W. Li, Y. Liu, and C. Liu, “Hand gesture recognition on a resource-limited interactive wristband,” *Sensors*, vol. 21, no. 17, p. 5713, Aug. 2021, doi: 10.3390/s21175713.
- [5] R. Zhang, R. Fang, M. Orooji, and H. Homayoun, “Introducing WeBe Band: an End-to-end Platform for Continuous Health Monitoring,” in *Proc. Annu. Int. Conf. IEEE Eng. Med. Biol. Soc. (EMBC)*, pp. 1–5, Jul. 2024, doi: 10.1109/EMBC53108.2024.10781832. <https://pubmed.ncbi.nlm.nih.gov/40040220>
- [6] I. Salehin, M. S. Islam, P. Saha, S. M. Noman, A. Tunj, M. M. Hasan, and M. A. Baten, “AutoML: A systematic review on automated machine learning with neural architecture search,” *J. Inf. Intell.*, vol. 2, no. 1, pp. 52–81, 2024, doi: 10.1016/j.jiixd.2023.10.002.
- [7] “Piccolo AI,” GitHub repository, 2024. [Online]. Available: <https://github.com/sensiml/piccolo>
- [8] L. van der Maaten and G. Hinton, “Visualizing data using t-SNE,” *Journal of Machine Learning Research*, vol. 9, pp. 2579–2605, 2008.
- [9] B. Hammer, B. Mokbel, F.-M. Schleif, and X. Zhu, “Prototype-based classification of dissimilarity data,” in *Advances in Intelligent Data Analysis X* (Lecture Notes in Computer Science, vol. 7014). Berlin, Heidelberg: Springer, 2011, pp. 185–197.
- [10] A. Kumar, S. Goyal, and M. Varma, “Resource-efficient machine learning in 2 KB RAM for the Internet of Things,” in *Proc. 34th Int. Conf. Mach. Learn. (ICML)*, Sydney, Australia, Aug. 6–11, 2017, vol. 70, pp. 1935–1944. [Online]. Available: <https://proceedings.mlr.press/v70/kumar17a.html>